

Experiment No: 5

Implement functions to print n^{th} Fibonacci number using iteration and recursive method. Compare the performance of two methods by counting number of steps executed on various inputs. Also draw a comparative chart. (Fibonacci series 1, 1, 2, 3, 5, 8..... Here 8 is the 6th Fibonacci number).

Date:

Competency and Practical Skills: Algorithmic thinking, Programming Skills, Performance analysis

Relevant CO: CO1, CO5

Objectives: (a) Compare the performances of two different versions of same problem.
(b) Find the time complexity of algorithms.
(C) Understand the polynomial and non-polynomial problems

Equipment/Instruments: Computer System, Any C language editor

Theory:

The Fibonacci series is the sequence of numbers (also called Fibonacci numbers), where every number is the sum of the preceding two numbers, such that the first two terms are '0' and '1'. In some older versions of the series, the term '0' might be omitted. A Fibonacci series can thus be given as, 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, . . . It can thus be observed that every term can be calculated by adding the two terms before it. We are ignoring initial zero in the series.

To represent any $(n+1)^{\text{th}}$ term in this series, we can give the expression as, $F_n = F_{n-1} + F_{n-2}$. We can thus represent a Fibonacci series as shown in the image below,

$$F(n) = \begin{cases} 0 & , n = 0 \\ 1 & , n = 1 \\ F(n-1) + F(n-2) & , n > 1 \end{cases}$$

Iterative version to print n^{th} Fibonacci number is as below:

Input: An integer n , where $n \geq 1$.

Output: The n^{th} Fibonacci number.

Steps:

Initialize variables $f_0 = 1$, $f_1 = 1$, and $i = 2$.

If n is 1 or 2 then

Print 1

While $i < n$, do

a. Set $f_2 = f_0 + f_1$.

b. Set $f_0 = f_1$.

c. Set $f_1 = f_2$.

d. Increment i by 1.

Print f_1 .

Recursive version to print n^{th} Fibonacci number is as below:

Input: An integer n, where $n \geq 1$.

Output: The n^{th} Fibonacci number.

If n is 1 or 2 then

return 1.

else recursively compute next number using the $(n-1)^{\text{th}}$ and $(n-2)^{\text{th}}$ Fibonacci numbers, and return their sum.

Print the result.

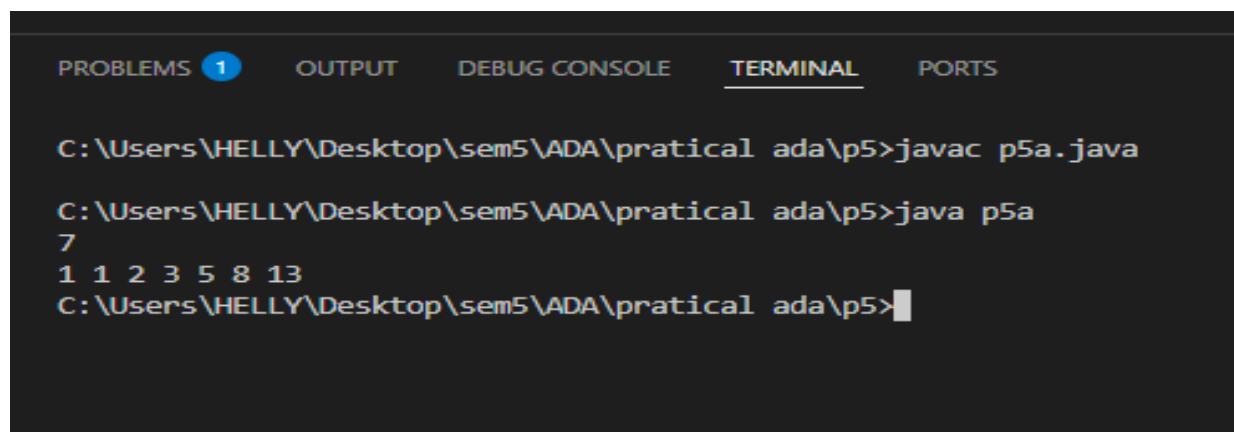
Implement functions of above two versions of Fibonacci series and compare the steps count of both the functions on various inputs ranging from 10 to 50 (if memory permits for recursive version).

Observations:

1) Iterative Method:

```
import java.util.Scanner;
```

```
public class p5a {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int n = sc.nextInt();  
        int f0 = 1, f1 = 1;  
        if (n >= 1)  
            System.out.print(f0 + " ");  
        if (n >= 2)  
            System.out.print(f1 + " ");  
        for (int i = 3; i <= n; i++) {  
            int f2 = f0 + f1;  
            System.out.print(f2 + " ");  
            f0 = f1;  
            f1 = f2;  
        }  
    }  
}
```



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS  
  
C:\Users\HELLY\Desktop\sem5\ADA\practical_ada\p5>javac p5a.java  
  
C:\Users\HELLY\Desktop\sem5\ADA\practical_ada\p5>java p5a  
7  
1 1 2 3 5 8 13  
C:\Users\HELLY\Desktop\sem5\ADA\practical_ada\p5>
```

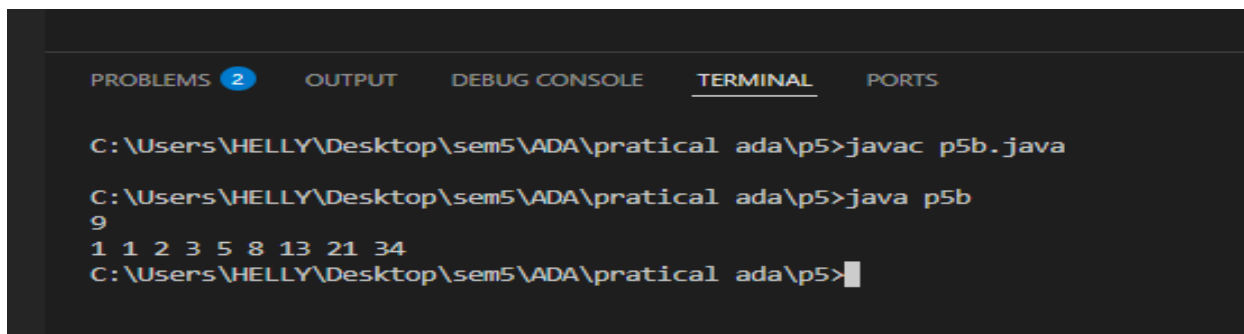
2) Recursive Method:

```
import java.util.Scanner;

public class p5b {

    public static int fibonacci(int n) {
        if (n <= 2)
            return 1;
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for (int i = 1; i <= n; i++) {
            System.out.print(fibonacci(i) + " ");
        }
    }
}
```



```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

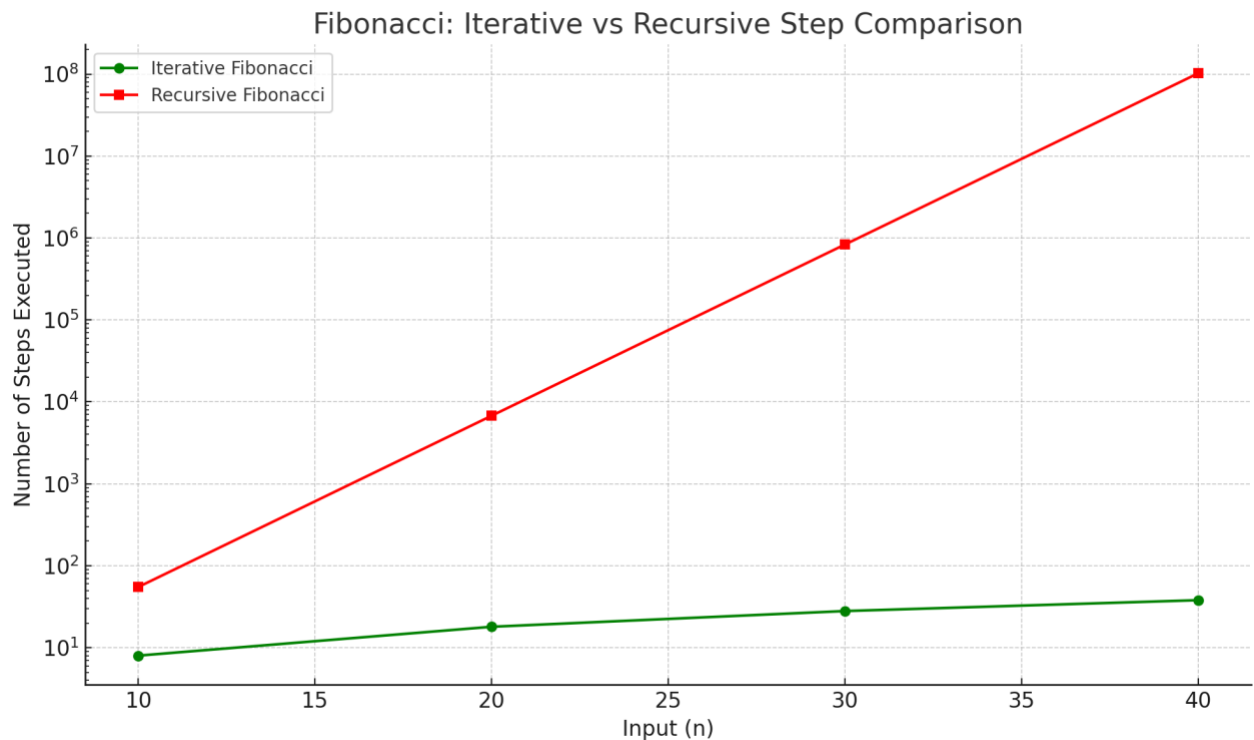
C:\Users\HELLY\Desktop\sem5\ADA\practical ada\p5>javac p5b.java

C:\Users\HELLY\Desktop\sem5\ADA\practical ada\p5>java p5b
9
1 1 2 3 5 8 13 21 34
C:\Users\HELLY\Desktop\sem5\ADA\practical ada\p5>
```

Result: Complete the below table based on your implementation of sequential search algorithm and steps executed by the function.

Inputs	Number of Steps Executed (Random data)	
	Iterative Fibonacci	Recursive Fibonacci
10	8	109
20	18	13,529
30	(if memory doesn't permit then reduce the range)	
40		
50		
Time Complexity?	$O(n)$	$O(2^n)$

Chart:



Conclusion:

- **Iterative Fibonacci** is **fast and efficient** ($O(n)$), suitable for large inputs.
- **Recursive Fibonacci** is **slow and inefficient** ($O(2^n)$), not practical without optimization.
- Use **iterative** method for performance; use recursive only for learning or with memoization.

Quiz:

1. What is the time complexity of iterative version of Fibonacci function?

Answer: $O(n)$

2. What is the time complexity of recursive version of Fibonacci function?

Answer: $O(2^n)$ (exponential time without memoization)

3. Can you execute recursive version of Fibonacci function for more inputs?

Answer: No, because it becomes very slow and may cause **stack overflow** or **memory issues** beyond $n \approx 35-40$, due to exponential growth in function calls.

4. What do you mean by polynomial time algorithms and exponential time algorithms?

Answer: Polynomial time algorithms run in time like $O(n)$, $O(n^2)$, $O(n^3)$ — the growth rate is manageable as input increases.

Exponential time algorithms grow like $O(2^n)$, $O(n!)$ — they become extremely slow for even modest input sizes, making them impractical.

Suggested Reference:

1. "Introduction to Algorithms" by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein
2. "Fundamentals of Algorithms" by E. Horowitz et al.

Rubric wise marks obtained:

Rubrics	Understanding of problem (3)			Program Implementation (5)			Documentation & Timely Submission (2)			Total (10)
	Good (3)	Avg. (2-1)	Poor (1-0)	Good (5-4)	Avg. (3-2)	Poor (1-0)	Good (2)	Avg. (1)	Poor (0)	
Marks										

Experiment No: 6

Implement a program for randomized version of quick sort and compare its performance with the normal version of quick sort using steps count on various inputs (1000 to 5000) of random nature, ascending order and descending order sorted data. Also, draw a comparative chart of number of input versus steps executed/time taken for each cases (random, ascending, and descending).

Date:

Competency and Practical Skills: Algorithmic thinking, Programming Skills, Performance analysis

Relevant CO: CO1, CO2

Objectives: (a) Improve the performance of quick sort in worst case.
(b) Compare the performance of both the version of quick sort on various inputs

Equipment/Instruments: Computer System, Any C language editor

Theory:

Steps to implement randomized version of quick sort are as below:

```
RANDOMIZED-QUICKSORT(A, low, high)
    if (low < high) {
        pivot = RANDOMIZED_PARTITION(A, low, high);
        RANDOMIZED-QUICKSORT(A, low, pivot);
        RANDOMIZED-QUICKSORT(A, pivot+1, high);
    }
RANDOMIZED_PARTITION (A, low, high) {
    pos = Random(low, high)
    pivot = A[pos]
    swap(pivot, A[low])
    left = low
    right = high
    while ( left < right ) {
        /* Move left while item < pivot */
        while( A[left] <= pivot ) left++;
        /* Move right while item > pivot */
        while( A[right] > pivot ) right--;
        if ( left < right )
            swap(A[left], A[right]);
    }
    /* right is final position for the pivot */
    swap(A[right], pivot);
    return right; }
```

Implement a function of randomized version of quick sort as per above instructions and use basic version of quick sort (that selects first element as pivot element). Compare the steps count of both the functions on various inputs ranging from 1000 to 5000 for each case (random, ascending, and descending).

Observations:

Write observation based on number of steps executed by both algorithms.

```
import java.util.Arrays;
import java.util.Random;

public class p6 {

    private static long basicComparisons;
    private static long basicSwaps;
    private static long randomizedComparisons;
    private static long randomizedSwaps;

    private static void swap(int[] arr, int i, int j, String type) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
        if (type.equals("basic")) {
            basicSwaps++;
        } else if (type.equals("randomized")) {
            randomizedSwaps++;
        }
    }

    private static int basicPartition(int[] arr, int low, int high) {
        int pivot = arr[low];
        int i = low;
        int j = high;

        while (i < j) {
            while (i <= high && arr[i] <= pivot) {
                basicComparisons++;
                i++;
            }
            while (j >= low && arr[j] > pivot) {
                basicComparisons++;
                j--;
            }
            if (i < j) {
                swap(arr, i, j, "basic");
            }
        }
        swap(arr, low, j, "basic");
        return j;
    }

    public static void basicQuickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pi = basicPartition(arr, low, high);
            basicQuickSort(arr, low, pi - 1);
            basicQuickSort(arr, pi + 1, high);
        }
    }
}
```

```

private static int randomizedPartition(int[] arr, int low, int high) {
    Random rand = new Random();
    int randomPivotIndex = low + rand.nextInt(high - low + 1);

    swap(arr, randomPivotIndex, low, "randomized");

    int pivot = arr[low];
    int i = low;
    int j = high;

    while (i < j) {
        while (i <= high && arr[i] <= pivot) {
            randomizedComparisons++;
            i++;
        }
        while (j >= low && arr[j] > pivot) {
            randomizedComparisons++;
            j--;
        }
        if (i < j) {
            swap(arr, i, j, "randomized");
        }
    }
    swap(arr, low, j, "randomized");
    return j;
}

public static void randomizedQuickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pi = randomizedPartition(arr, low, high);
        randomizedQuickSort(arr, low, pi - 1);
        randomizedQuickSort(arr, pi + 1, high);
    }
}

private static void generateRandomArray(int[] arr) {
    Random rand = new Random();
    for (int i = 0; i < arr.length; i++) {
        arr[i] = rand.nextInt(10000);
    }
}

private static void generateAscendingArray(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        arr[i] = i;
    }
}

private static void generateDescendingArray(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        arr[i] = arr.length - 1 - i;
    }
}

```



```

}

public static void main(String[] args) {
    int[] inputSizes = {1000, 2000, 3000, 4000, 5000};

    System.out.println("--- Performance Comparison of Quick Sort Variants (Java) ---\n");

    System.out.println("Random Data:");
    System.out.println("Input Size | Randomized Quick Sort (Steps) | Basic Quick Sort (Steps)");
    System.out.println("-----");
    runExperiment(inputSizes, "random");
    System.out.println();

    System.out.println("Ascending Data:");
    System.out.println("Input Size | Randomized Quick Sort (Steps) | Basic Quick Sort (Steps)");
    System.out.println("-----");
    runExperiment(inputSizes, "ascending");
    System.out.println();

    System.out.println("Descending Data:");
    System.out.println("Input Size | Randomized Quick Sort (Steps) | Basic Quick Sort (Steps)");
    System.out.println("-----");
    runExperiment(inputSizes, "descending");
    System.out.println();
}

private static void runExperiment(int[] inputSizes, String dataType) {
    for (int size : inputSizes) {
        int[] arrRandomized = new int[size];
        int[] arrBasic = new int[size];

        switch (dataType) {
            case "random":
                generateRandomArray(arrRandomized);
                break;
            case "ascending":
                generateAscendingArray(arrRandomized);
                break;
            case "descending":
                generateDescendingArray(arrRandomized);
                break;
        }

        System.arraycopy(arrRandomized, 0, arrBasic, 0, size);

        basicComparisons = 0;
        basicSwaps = 0;
        randomizedComparisons = 0;
        randomizedSwaps = 0;

        randomizedQuickSort(arrRandomized, 0, size - 1);
        long randomizedTotalSteps = randomizedComparisons + randomizedSwaps;
    }
}

```

```

        basicQuickSort(arrBasic, 0, size - 1);
        long basicTotalSteps = basicComparisons + basicSwaps;

        System.out.printf("%10d | %30d | %30d%n", size, randomizedTotalSteps, basicTotalSteps);
    }
}
}

```

```

[Done] exited with code=1 in 3.159 seconds

[Running] cd "c:\Users\HELLY\Desktop\sem5\ADA\practical_ada\p6\" && javac p6.java && java p6
--- Performance Comparison of Quick Sort Variants (Java) ---

Random Data:
Input Size | Randomized Quick Sort (Steps) | Basic Quick Sort (Steps)
-----
1000 | 14164 | 13675
2000 | 32833 | 32183
3000 | 54375 | 48781
4000 | 72961 | 64981
5000 | 92102 | 88181

Ascending Data:
Input Size | Randomized Quick Sort (Steps) | Basic Quick Sort (Steps)
-----
1000 | 12851 | 501498
2000 | 27583 | 2002998
3000 | 46436 | 4504498
4000 | 66162 | 8005998
5000 | 79702 | 12507498

Descending Data:
Input Size | Randomized Quick Sort (Steps) | Basic Quick Sort (Steps)
-----
1000 | 12771 | 501498
2000 | 28491 | 2002998
3000 | 45858 | 4504498
4000 | 64988 | 8005998
5000 | 81143 | 12507498

[Done] exited with code=0 in 22.129 seconds

```

Result: Complete the below table based on your implementation of sequential search algorithm and steps executed by the function.

Inputs	Number of Steps Executed (Random data)	
	Randomized Quick Sort	Basic Quick Sort
1000	14693	14660
2000	31514	29499
3000	49111	50412
4000	71070	65127
5000	88129	93305
Time Complexity?	$O(n \log n)$	$O(n \log n)$

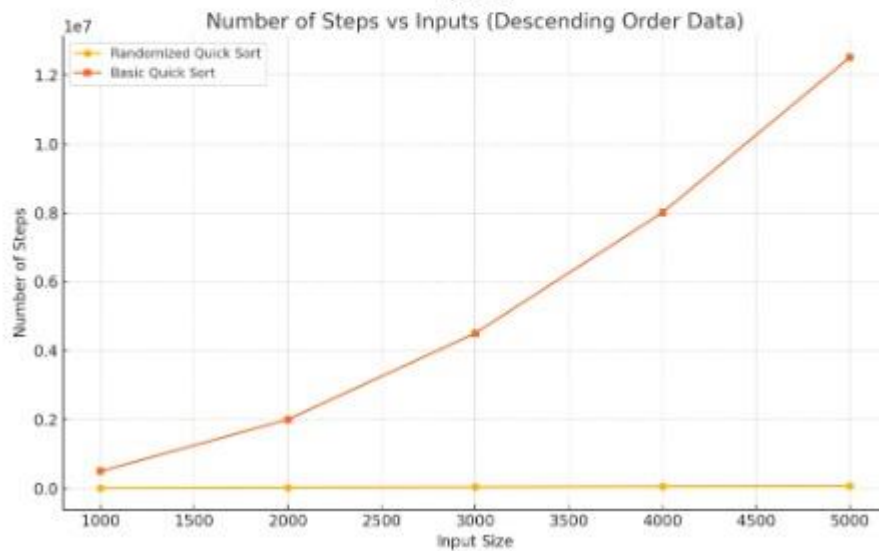
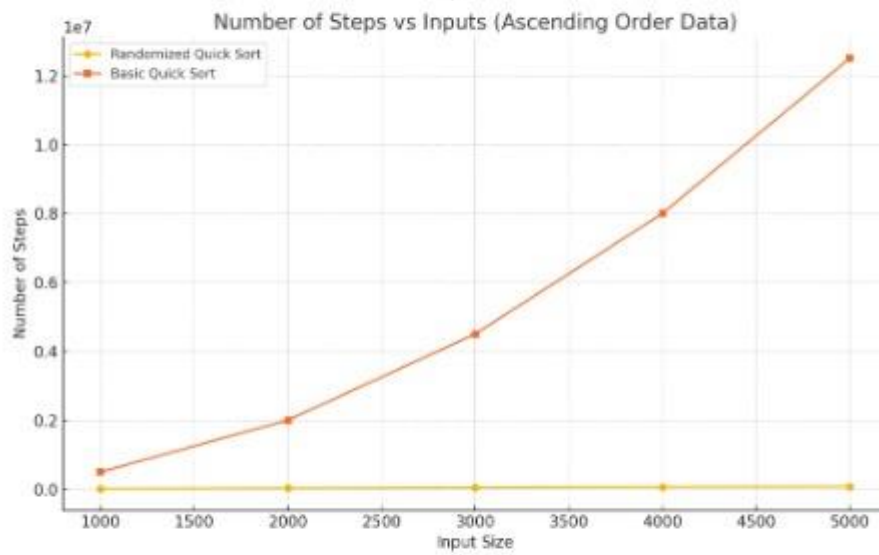
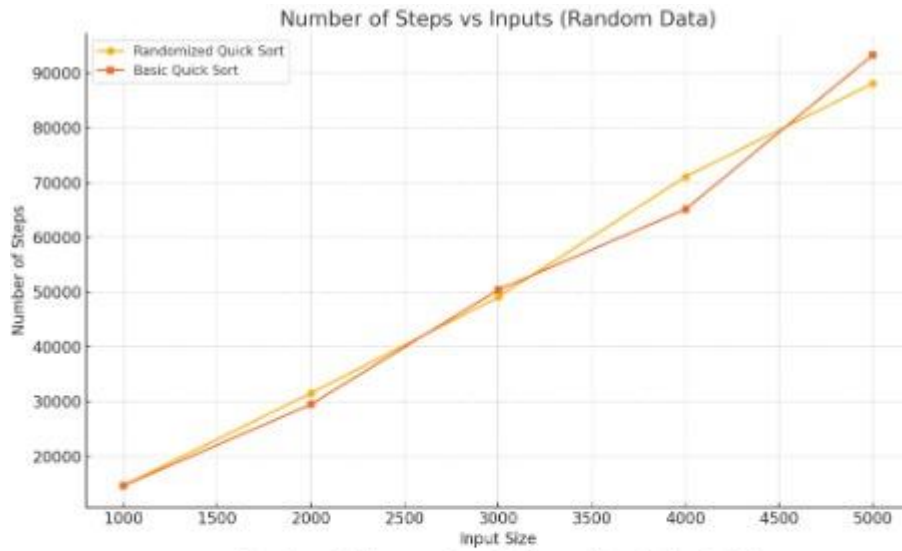
-descending order :

Inputs	Number of Steps Executed dec order)	
	Randomized Quick Sort	Basic Quick Sort
1000	12316	501498
2000	28402	2002998
3000	43804	4504498
4000	62104	8005998
5000	78577	12507498
Time Complexity□	$O(n \log n)$	$O(n \log n)$

-ascending order sorted data:

Inputs	Number of Steps Executed (asc order)	
	Randomized Quick Sort	Basic Quick Sort
1000	14777	5014998
2000	29649	2002998
3000	46976	4504498
4000	66503	8005998
5000	83126	12507498
Time Complexity□	$O(n \log n)$	$O(n \log n)$

Chart:



Rubrics	Understanding of problem (3)			Program Implementation (5)			Documentation & Timely Submission (2)			Total (10)
	Good (3)	Avg. (2-1)	Poor (1-0)	Good (5-4)	Avg. (3-2)	Poor (1-0)	Good (2)	Avg. (1)	Poor (0)	
Marks										

Experiment No: 7

Implement program to solve problem of making a change using dynamic programming.

Date:

Competency and Practical Skills: Algorithmic thinking, Programming Skills, Problem solving

Relevant CO: CO3

Objectives: (a) Understand Dynamic programming algorithm design method.
(b) Solve the optimization based problem.
(c) Find the time complexity of the algorithm.

Equipment/Instruments: Computer System, Any C language editor

Theory:

Making Change problem is to find change for a given amount using a minimum number of coins from a set of denominations. If we are given a set of denominations $D = \{d_0, d_1, d_2, \dots, d_n\}$ and if we want to change for some amount N , many combinations are possible. Suppose $\{d_1, d_2, d_5, d_8\}$, $\{d_0, d_2, d_4\}$, $\{d_0, d_5, d_7\}$ all are feasible solutions but the solution which selects the minimum number of coins is considered to be an optimal solution. The aim of making a change is to find a solution with a minimum number of coins / denominations. Clearly, this is an optimization problem.

General assumption is that infinite coins are available for each denomination. We can select any denomination any number of times.

Solution steps are as follow:

Sort all the denominations and start scanning from smallest to largest denomination. In every iteration i , if current denomination d_i is acceptable, then 1 coin is added in solution and total amount is reduced by amount d_i . Hence,

$$C[i, j] = 1 + (c[i, j - d_i])$$

$C[i, j]$ is the minimum number of coins to make change for the amount j . Below figure shows the content of matrix C .

	j												
	0	1	2	3	4	5	6	7	8	9	10	11	12
1	0	1	2	3	4	5	1	2	3	4	1	2	2
2	0	1	2	3	4	5	1	2	3	4	5	6	2
3	0	1	2	3	4	5	6	7	8	9	10	11	12

Figure: Content of matrix C

using coins if current denomination is larger than current problem size, then we have to skip the denomination and stick with previously calculated solution. Hence,

$$C[i, j] = C[i - 1, j]$$

If above cases are not applicable then we have to stick with choice which returns minimum number of coin. Mathematically, we formulate the problem as,

$$C[i, j] = \min \{C[i - 1, j], 1 + C[i, j - d_i]\}$$

Steps to solve making change problem are as below:

Algorithm MAKE_A_CHANGE(d,N)
 // d[1...n] = [d1,d2,...,dn] is array of n denominations
 // C[1...n, 0...N] is n x N array to hold the solution of sub problems
 // N is the problem size, i.e. amount for which change is required

```

for i ← 1 to n do
    C[i, 0] ← 0
end
for i ← 1 to n do
    for j ← 1 to N do
        if i == 1 and j < d[i] then
            C[i, j] ← ∞
        else if i == 1 then
            C[i, j] ← 1 + C[1, j - d[1]]
        else if j < d[i] then
            C[i, j] ← C[i - 1, j]
        else
            C[i, j] ← min (C[i - 1, j], 1 + C[i, j - d[i]])
        end
    end
end
return C[n, N]
```

Implement above algorithm and print the matrix C. Your program should return the number of coins required and its denominations.

Observations:

Write observation based on whether this algorithm returns optimal answer or not on various inputs.

```
import java.util.Arrays;
```

```
public class CoinChange {
```

```
    public static int makeAChange(int[] d, int N) {
        int n = d.length;
        int[][] C = new int[n][N + 1];
```

```
        for (int i = 0; i < n; i++) {
            C[i][0] = 0;
        }
```

```
        for (int i = 0; i < n; i++) {
            for (int j = 1; j <= N; j++) {
                if (i == 0 && j < d[i]) {
```

```

        C[i][j] = Integer.MAX_VALUE - 1;
    } else if (i == 0) {
        if (C[i][j] - d[i] != Integer.MAX_VALUE - 1)
            C[i][j] = 1 + C[i][j] - d[i];
        else
            C[i][j] = Integer.MAX_VALUE - 1;
    } else if (j < d[i]) {
        C[i][j] = C[i - 1][j];
    } else {
        if (C[i][j] - d[i] != Integer.MAX_VALUE - 1)
            C[i][j] = Math.min(C[i - 1][j], 1 + C[i][j] - d[i]);
        else
            C[i][j] = C[i - 1][j];
    }
}
}

return C[n - 1][N] == Integer.MAX_VALUE - 1 ? -1 : C[n - 1][N];
}

public static void main(String[] args) {
    int[] denominations = {1, 3, 4};
    int amount = 6;
    int result = makeAChange(denominations, amount);
    System.out.println("Minimum number of coins required: " + result);
}
}

```

Result:

The screenshot shows a terminal window with the following content:

```

practical ada\p7>javac p7a.java
C:\Users\HELLY\Desktop\sem5\ADA\practical ada\p7>java p7a
Minimum number of coins required: 2
C:\Users\HELLY\Desktop\sem5\ADA\practical ada\p7>

```

A tooltip is visible over the first command, stating: "Focus folder in explorer (ctrl + click)".

Conclusion: • The **dynamic programming approach** efficiently solves the coin change problem by **breaking it into subproblems** and building up solutions.

- The algorithm ensures **minimum coins** are used for any target amount.

Quiz:

1. What is the time complexity of above algorithm?

Answer: $O(n \times N)$

2. Does above algorithm always return optimal answer?

Answer: Yes , The algorithm uses **dynamic programming** to explore all possibilities and always returns the **minimum number of coins** needed to make the target amount.

Suggested Reference:

1. "Introduction to Algorithms" by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein
2. "Fundamentals of Algorithms" by E. Horowitz et al.
3. <https://codecrucks.com/making-change-problem-using-dynamic-programming/>

Rubric wise marks obtained:

Rubrics	Understanding of problem (3)			Program Implementation (5)			Documentation & Timely Submission (2)			Total (10)
	Good (3)	Avg. (2-1)	Poor (1-0)	Good (5-4)	Avg. (3-2)	Poor (1-0)	Good (2)	Avg. (1)	Poor (0)	
Marks										